Introduction

2026-1학기 부경대학교 알고리즘 스터디

Problem Solving(PS) 이란 무엇인가?

- 문제해결: 주어진 문제를 제한된 시간 및 메모리 안에서 해결하는 것을 뜻함
- 알고리즘과 자료구조 등을 사용하여 최적화해야 효율적으로 해결
- 머릿속에서 나온 풀이가 정당한지도 생각하기

Problem Solving 입문

- 다양한 문제풀이 사이트들이 존재하고, 이러한 사이트에서 문제풀이 연습을 할 수 있음.
- Online Judge(OJ): 문제 은행 방식으로 시간에 구애 받지 않고 문제풀이가 가능
  - 국내에서는 백준, 프로그래머스를 가장 많이 애용
  - 해당 스터디에서는 백준을 이용할 예정
- Online Contest: 경쟁적 프로그래밍 대회를 온라인으로 참여할 수 있음
  - Competitive Programming(CP): 제한 시간 안에 남들보다 최대한 많은 문제를 풀어내는 방식의 프로그래밍 방식
  - 유명한 사이트로 Codeforces(러시아), AtCoder(일본)가 있음.

스터디의 목적

- 기업 코딩테스트 전반으로 나오는 대부분의 알고리즘 익히기
- 문제풀이를 연습하면서 실질적인 문제풀이 감각 익히기
- 한 학기동안 PS로 열심히 불태워보자

스터디 구성

주차	주제	세부 내용
1	OT, 시간복잡도	Time Complexity, Brute Force
2	기초 자료구조	Data Structure(Stack, Queue, Deque, Set, Map)
3	정렬과 탐색	Sort, Binary Search, Two Pointer
4	재귀와 활용	Recursion, Backtracking, Divide and Conquer, Master Theorem
5	동적계획법과 그리디	DP(Prefix Sum, Knapsack, LIS), Greedy
6	그래프와 순회	Graph Theory, Graph Traversal(DFS, BFS)
7	방향성 비순환 그래프	DAG, Topological Sort
8	트리와 우선순위 큐	Graph Theory in Tree, Priority Queue
9	최소신장트리	Union Find, MST, Kruskal, Prim
10	최단경로알고리즘	Dijkstra, Bellman Ford, Floyd Warshall
부록 1	PS를 위한 수학	GCD, LCM, Eratos Sieve, Permutation, Combination, Primality Test
부록 2	PS를 위한 C++	class and object, namespace, iostream, STL, range-based for loops, reference, <bits/stdc++.h>

Time Complexity

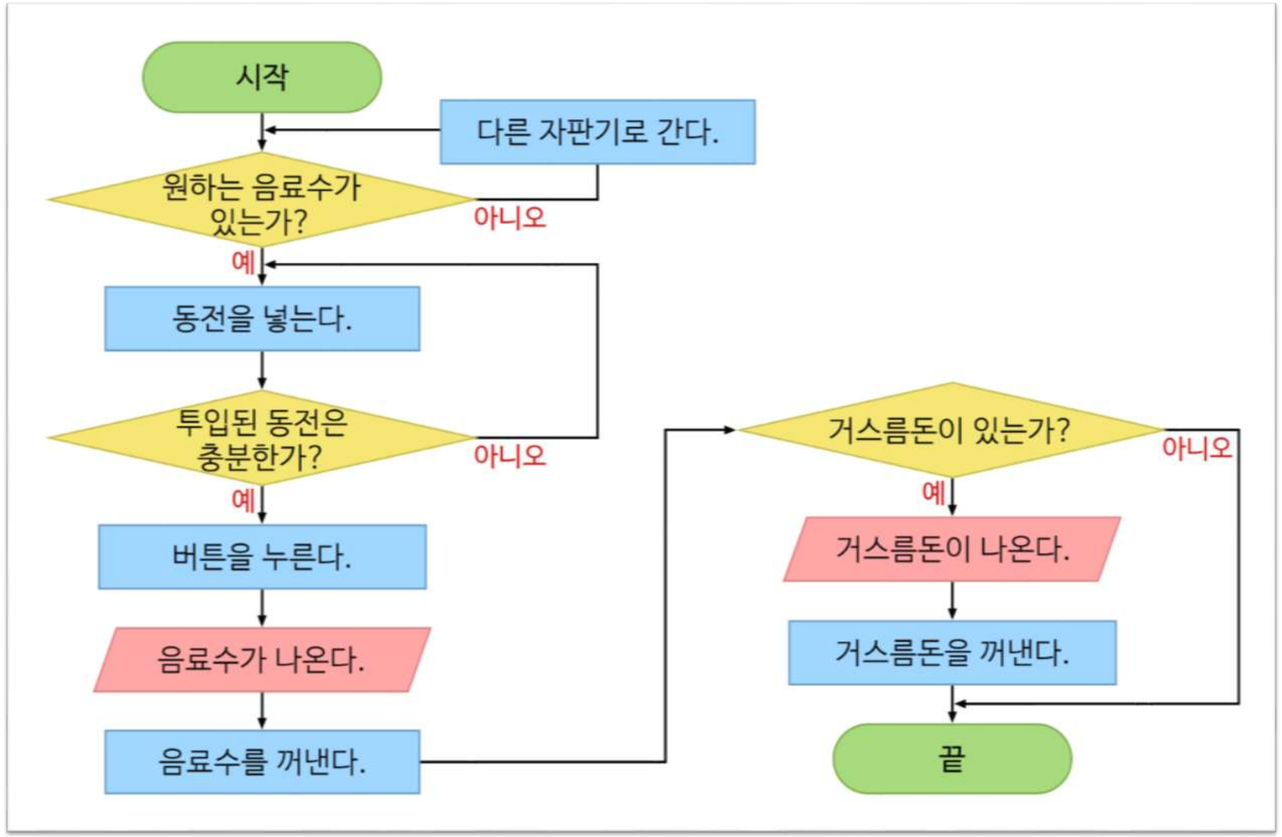
2026-1학기 부경대학교 알고리즘 스터디

목차

- 알고리즘이란?
- 알고리즘 성능의 척도
- 시간 복잡도
- 점근표기법
- 시간 복잡도 분석

알고리즘이란?

- 문제를 해결하기 위한 일련의 과정을 뜻함.
  - ex) 자판기에서 원하는 음료를 뽑는 과정



알고리즘이란?

- 알고리즘의 필수 특징
  - 입력(Input): 0개 이상의 외부에서 제공된 자료가 존재 해야함.
  - 출력(Output): 수행 이후 1개 이상의 결과를 가져야 함.
  - 명확성(Definiteness): 모든 단계들은 모호하지 않고 명확해야 함.
  - 유한성(Finiteness): 유한 시간 내에 종료되어야 함.
  - 유효성(Effectiveness): 모든 명령들은 수행가능해야 함.

알고리즘이란?

- 알고리즘의 표현 방법
  - 자연어: 사람이 일상적으로 사용하는 언어
  - 순서도: 일련의 과정을 간단한 도형과 화살표로 도식화한 그림
  - 의사코드: 자연어를 프로그래밍 언어와 유사한 방식으로 배치한 것
  - 프로그래밍 언어: 컴퓨터에게 명령 및 연산을 시킬 목적으로 만들어진 언어

알고리즘 성능의 척도

- 알고리즘의 성능을 평가하는 대표적인 기준은 크게 5가지로 나눌 수 있음
  - 작업량: 기능을 수행하는데 얼마나 적은 작업이 필요한가
  - 메모리 사용량: 실행 시 메모리를 얼마나 적게 차지하는가
  - 정확성: 실행하여 나온 결과가 얼마나 정확한가
  - 단순성: 작동과정이 얼마나 단순한가
  - 최적성: 더 이상 개선의 여지가 없을 만큼 최적화가 잘 되어 있는가

※ 위 네가지 성능의 척도를 다 포함하는 포괄적인 척도. 사용자의 판단과 문제 상황에 따라 기준이 달라질 수 있음
※ PS에서는 작업량의 개선을 나타내는 경우가 대부분

알고리즘 성능의 척도

- 작업량이 많을 경우 -> 시간 초과
- 메모리를 한번에 많이 사용할 경우 -> 메모리 초과
- 결과를 정확하게 도출하지 못할 경우 -> 틀렸습니다
- 단순하지 않을 경우 -> 구현이 어려움

시간 복잡도

- 정의: 입력의 크기와 실행속도의 관계를 나타내는 함수
  - 알고리즘의 작업량을 나타내는 요소 중 하나
  - CPU 점유, 스로틀링, 네트워크 등의 부수적 요소는 신경 쓰지 않음
  - 대부분 최악의 경우를 계산함

시간 복잡도

- 예시 1: 배열 A의 크기가 N이고, 입력으로 양의 정수 K가 주어졌을 때 배열에 K가 있는지 확인
  - 입력: 양의 정수 N과 K, 배열 A에 속하는 N개의 원소들
  - 기본 연산 정의: 배열의 한 원소가 K인지 확인
  - 알고리즘의 설계: A[0]부터 A[N-1]까지 순차적으로 확인하고, 도중에 K가 있으면 중단한다.
  - 기본 연산의 최대 횟수: 최대 N회
  - 시간 복잡도 $T(N) = N$ (최악의 경우)

```
bool checkArray(int N, int K, int A[]) {
  for (int i = 0; i < N; i++) {
    if (A[i] == K) return true;
  }
  return false;
}
```

시간 복잡도

- 예시 2: 크기가 N인 배열 A가 주어졌을 때 버블 정렬 알고리즘으로 정렬
  - 버블 정렬: A[0]과 A[1] 비교하여 정렬하고 A[1]와 A[2] 비교하여 정렬하고 A[2]와 A[3] ... A[N-2]와 A[N-1] 비교하여 정렬 이후 A[0]으로 돌아와 A[N-2]까지 반복, A[N-3]까지 반복... A[2]까지 반복
  - 입력: N과 크기가 N인 배열 A
  - 기본 연산: 두 원소를 비교하여 값 뒤바꾸기
  - 기본 연산의 최대 횟수: $(N-1) + (N-2) + \cdots + 2 + 1 = N(N-1)/2$번
  - 시간 복잡도 $T(N) = \frac{N(N-1)}{2}$

```
void bubbleSort(int N, int A[]) {
  for (int i = 0; i < N; i++) {
    for (int j = i + 1; j < N; j++) {
      if (A[j - 1] > A[j]) swap(A[j - 1], A[j]);
    }
  }
}
```

시간 복잡도

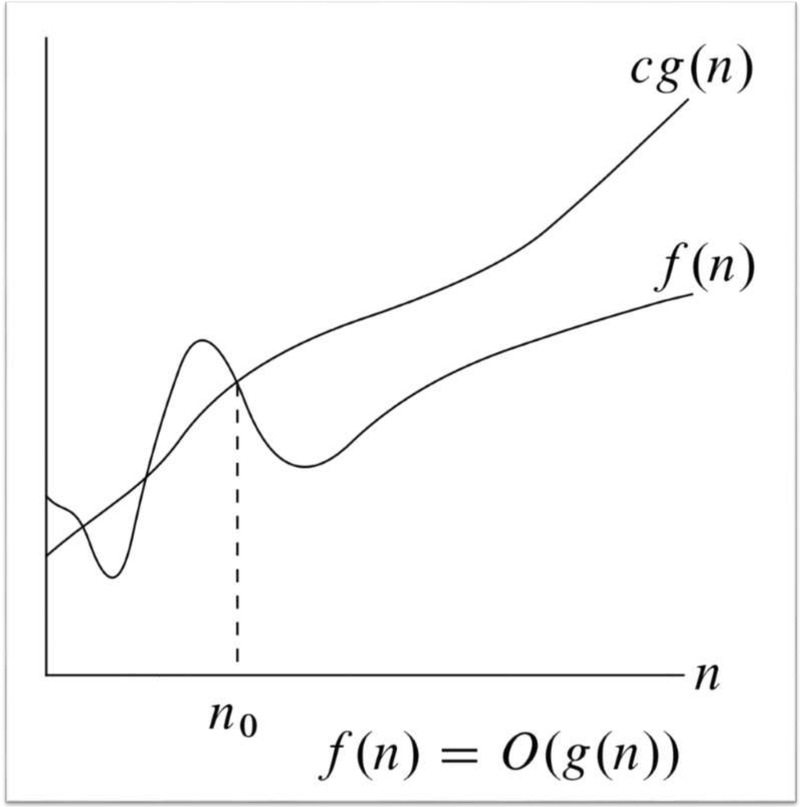
- 왜 대부분의 시간 복잡도를 최악의 경우로 계산할까?
  - 시스템 설계에서는 신뢰성이 정말 중요
  - 이 신뢰성이란 “성능”으로서의 신뢰성도 포함
  - 만약 알고리즘이 예상된 작업 시간보다 오래 걸린다면 -> 신뢰도 하락
  - 최악의 경우를 계산하면 작업 시간의 상한선이 보장됨 -> 신뢰성의 보장으로 이어짐

점근 표기법

- 알고리즘의 시간 복잡도를 대략적으로 나타내는 방법 (漸[차츰 점] 近[가까울 근])
  - 시간 복잡도 뿐만 아니라 공간 복잡도 또한 해당 방식으로 나타낼 수 있음.
- 총 세 가지 표기법이 존재
  - Big-O: 수행시간의 상한 (최악)
  - Big-Theta: 수행시간의 하한 (최선)
  - Big-Omega: 수행시간의 상한+하한

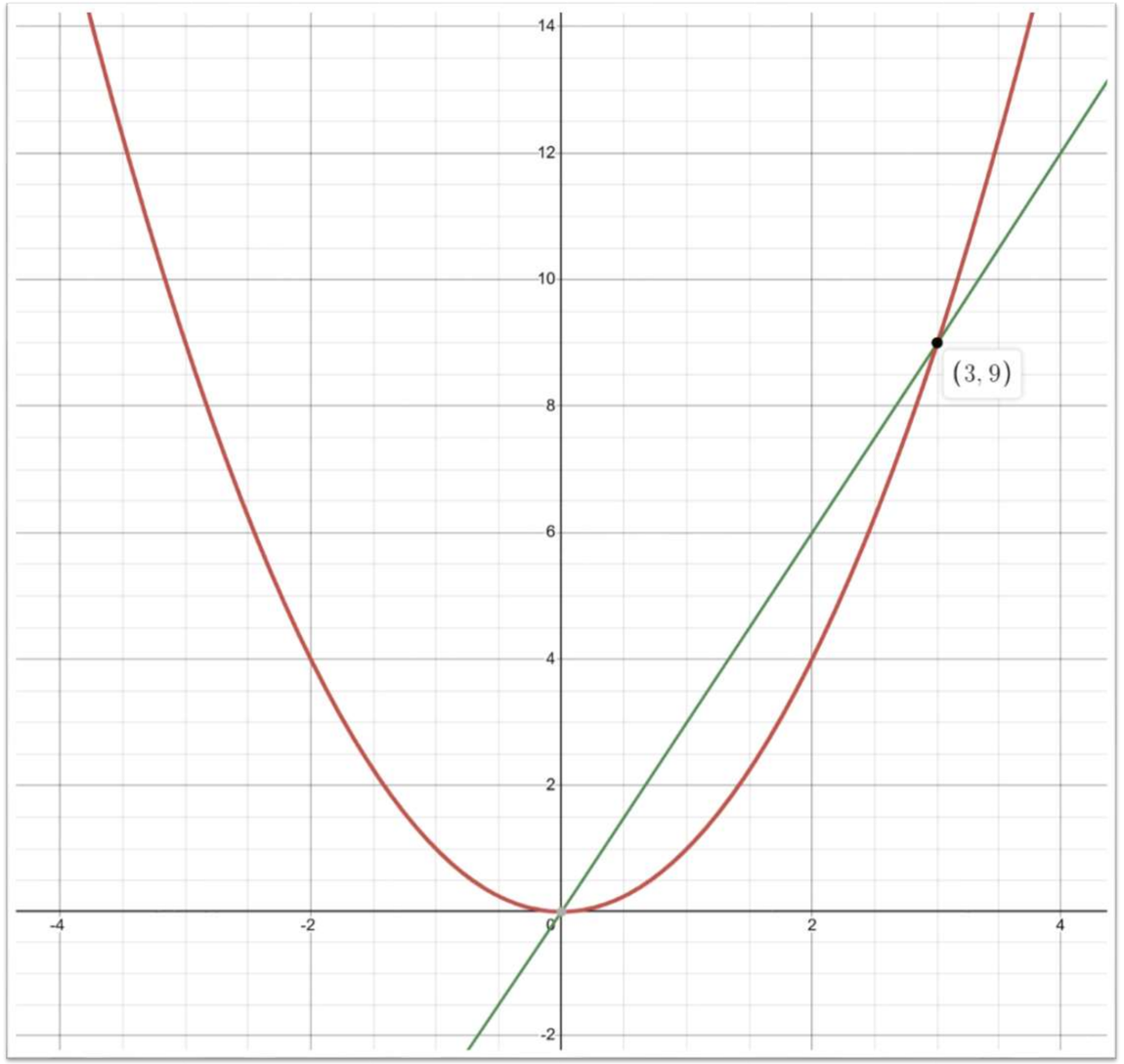
점근 표기법

- Big-O 표기법
  - 입력 크기에 대해 수행시간의 상한을 나타내는 함수
  - 정의: 실함수 $f, g: [0, \infty) \to \mathbb{R}$에 대해 양수 n_0, c가 존재하여 $n > n_0$에 대해 $f(n) \leq cg(n)$을 만족하면 $f(n) \in O\big(g(n)\big)$
  - 쉽게 이야기 하여 일정 시점(n_0)부터 계속 $f(n) \leq cg(n)$이면 $f(n) \in O(g(n))$



점근 표기법

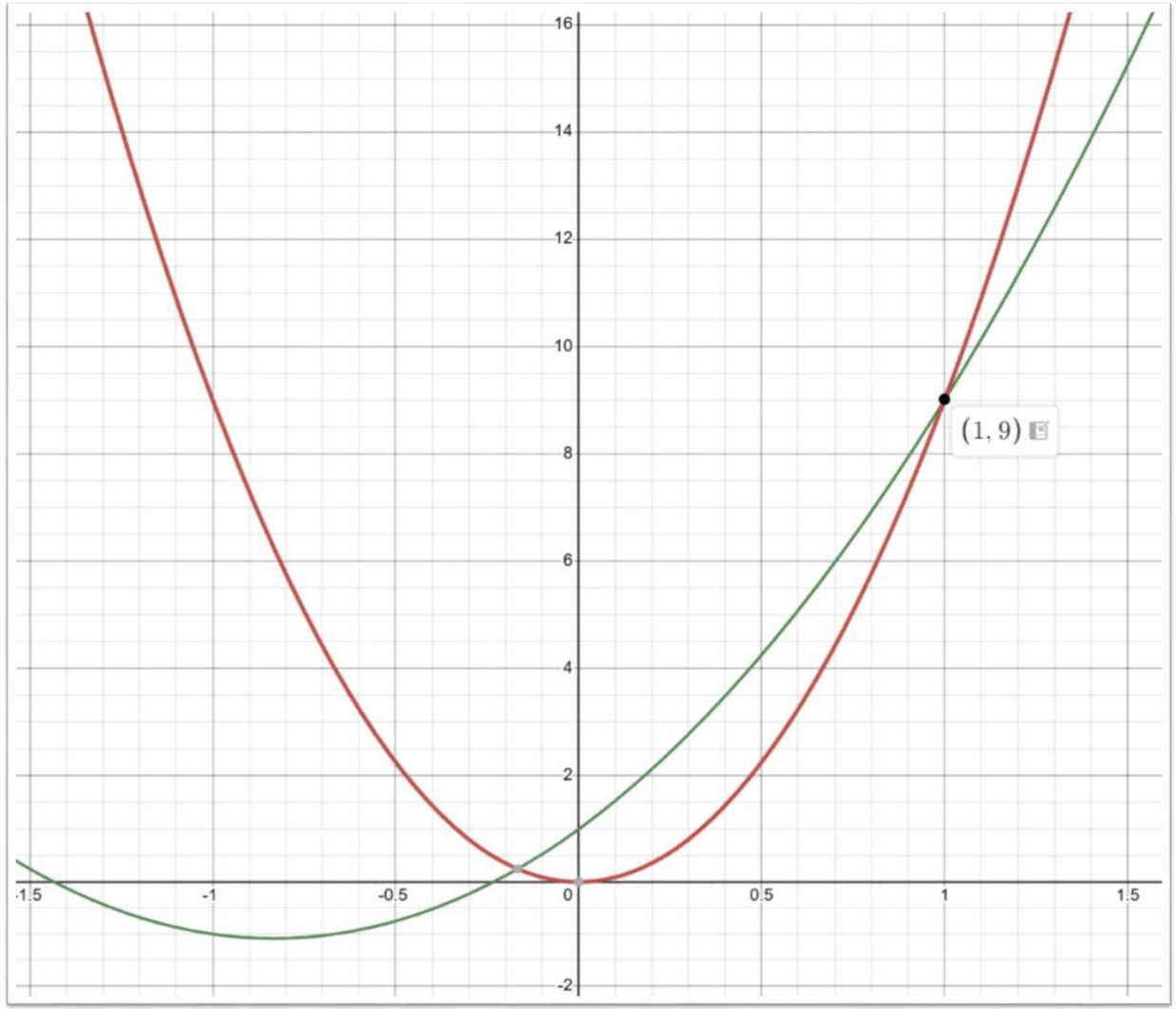
- 예시 1. $f(n) = 3n, g(n) = n^2$
  - $n_0 = 3, c = 1$ 이면 $3n^2 \leq n^2$
  - 따라서 $3n \in O(n^2)$



$f(n) = 3n, c \cdot g(n) = 1 \cdot n^2$ 이면, $n > 3 \rightarrow f(n) \leq cg(n)$

점근 표기법

- 예시 2. $f(n) = 3n^2 + 5n + 1, g(n) = n^2$
  - $n_0 = 1, c = 9$ 이면 $3n^2 + 5n + 1 \leq 9n^2$
  - 따라서 $3n^2 + 5n + 1 \in O(n^2)$



$f(n) = 3n^2 + 5n + 1, c \cdot g(n) = 9 \cdot n^2$ 이면, $n > 1 \rightarrow f(n) \leq cg(n)$

점근 표기법

- 예시 3. $f(n) = 5, g(n) = 1$
  - $n_0 = 0, c = 6$ 이면 $5 \leq 6$
  - 따라서 $5 \in O(1)$

$f(n) = 5, c \cdot g(n) = 6 \cdot 1$ 이면, $n > 0 \rightarrow f(n) \leq cg(n)$

점근 표기법

- Big-O 표기법 쉽게 구하는 방법?
- 다항 함수: 최고차항만 제외하고 없애기
  - n값이 커지면 커질수록 차수가 낮은 항들은 무시가 가능할 정도로 비중이 점점 적어짐
  - ex) $n^3 + 100n^2 + 10 \in O(n^3)$
- 지수 함수: 밑이 가장 큰 항만 제외하고 없애기
  - 마찬가지로 n값이 커지면 커질수록 밑이 작은 항들은 무시 가능할 정도로 비중이 적어짐
  - 이 때 다항함수를 포함하여 없앤다.
  - ex) $3^n + 2^n + 8n^3 + 2n + 1 \in O(3^n)$

점근 표기법

- Big-O 표기법의 위계
  - $O(1) \in O(\log\log n) \in O\left(\log^k n\right) \in O(\sqrt{n}) \in O\left(n^k\right) \in O(2^n) \in O(3^n) \in O(n!) \in O(n^n)$

    ※ 단, k는 양의 정수

    ※ $\log\log n = \log(\log n)$이고, $\log^2 n = \log n \cdot \log n$을 뜻함
  - 대략적인 것만 간추려서 그렇지, 실제 복잡도는 굉장히 다양하게 나옴.
  - 위계에 따라서 제일 큰 항만 제외하고 삭제하기. ex) $O(n + \sqrt{\mathrm{n}} + \log\log \mathrm{n}) = O(n)$
- 착각할 수 있는 사항
  - Big-O 표기법에서 임의의 복잡도보다 낮은 복잡도를 가진 함수는 전부 해당 복잡도에 포함됨
  - ex) $f(n) = n^2$라고 할 때, $f(n) \in O(n^2)$도 되고, $f(n) \in O(n^3)$, $f(n) \in O(2^n)$등도 가능함.

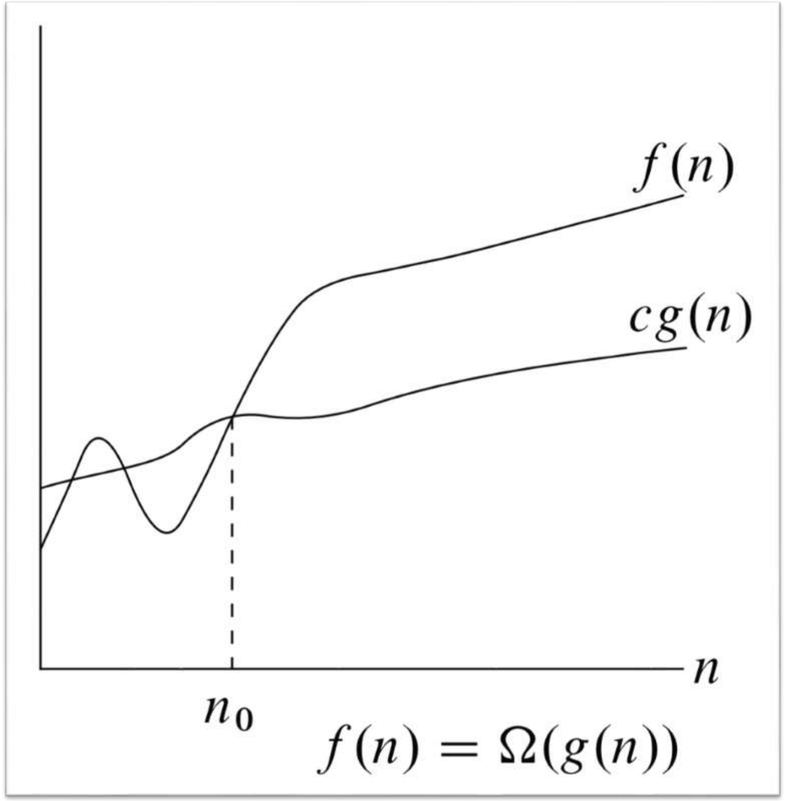
점근 표기법

- 다변수 함수로도 사용
  - 정의: 다변수 실함수 $f, g: \mathbb{R}^2 \to \mathbb{R}$에 대해 양수 n_0, m_0, c가 존재하여

    $n > n_0, m > m_0$에 대해 $f(n,m) \le cg(n,m)$을 만족하면 $f(n,m) \in O\big(g(n,m)\big)$
  - 마찬가지로 최고차항만 남기고 없애나, 대소를 비교할 수 없는 항의 경우는 남겨두기
  - ex) $n^2 + 3nm + 6m^2 + 4 = O(n^2 + nm + m^2)$

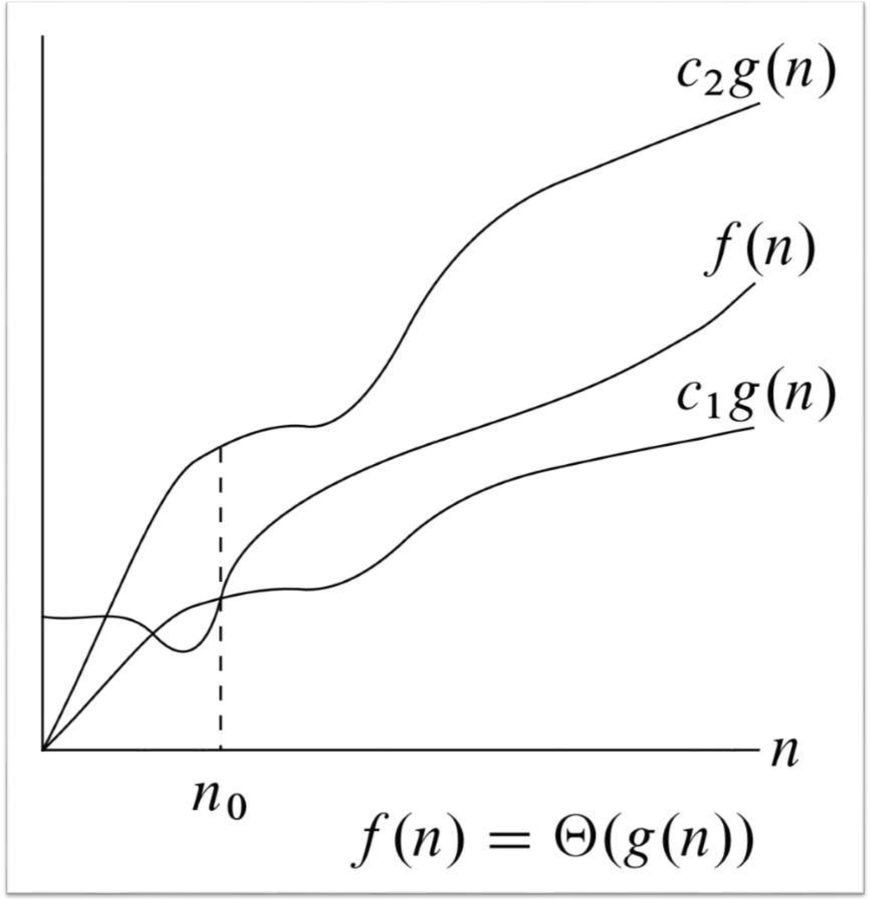
점근 표기법

- Big-Omega 표기법
  - 입력 표기에 대해 하한을 나타내는 함수 (Big-O와 반대)
  - 정의: 실함수 $f, g: [0, \infty) \to \mathbb{R}$에 대해 양수 n_0, c가 존재하여 $n > n_0$에 대해 $f(n) \geq cg(n)$을 만족하면 $f(n) \in \Omega\big(g(n)\big)$
  - 쉽게 이야기 하여 일정 시점(n_0)부터 계속 $f(n) \geq cg(n)$이면 $f(n) \in \Omega(g(n))$
- 즉, 증가율이 같거나, 더 높은 함수
  - $3n + 1 \notin \Omega(n^2)$
  - $5n^2 + 3n \in \Omega(n^2)$
  - $n^3 \in \Omega(n^2)$



점근 표기법

- Big-Theta 표기법
  - 입력 크기에 대해 상한과 하한을 동시에 나타내는 함수 (교집합)
  - 정의: 실함수 $f, g: [0, \infty) \to \mathbb{R}$에 대해 양수 $n_0, c,_1 c_2$가 존재하여 $n > n_0$에 대하여 $f(n) \geq c_1 g(n), f(n) \leq c_2 g(n)$을 만족하면 $f(n) \in \Theta\big(g(n)\big)$
- 즉, 증가율이 같은 함수를 뜻함
  - $3n + 1 \notin \Theta(n^2)$



점근 표기법

- 주의사항
  - 점근 표기법은 알고리즘의 입력크기에 따른 수행시간의 증가비율을 "n에 비례적"으로 나타낸 것.
  - 절대적으로 걸리는 시간이 아님.
  - Big-O 표기법: 아무리 많은 시간이 걸리더라도 최대 수행시간은 대략 이 정도.
  - Big-Omega 표기법: 아무리 적은 시간이 걸리더라도 최소 수행시간이 대략 이 정도.
  - Big-Theta 표기법: 해당 알고리즘의 수행 시간은 많거나 적거나 대략 이 정도.

시간 복잡도 분석

- 버블 정렬 알고리즘의 시간 복잡도를 Big-O 표기법으로 계산해보자.
  - 버블 정렬의 총 연산 횟수는 $T(N) = \frac{N(N-1)}{2} = \frac{N^2}{2} - \frac{N}{2} \in O(\mathrm{N}^2)$
  - 따라서 버블 정렬의 시간 복잡도는 $O(N^2)$
  - 허나 총 연산 횟수를 정확하게 계산하기 어려운 경우도 많음
  - 이런 경우 -> 안쪽 for문이든, 바깥쪽 for문이든 계산 횟수는 전부 $O(N)$이므로, $O(N) \cdot O(N) = O(N^2)$

```
void bubbleSort(int N, int A[]) {
  for (int i = 0; i < N; i++) {
    for (int j = i + 1; j < N; j++) {
      if (A[j - 1] > A[j]) swap(A[j - 1], A[j]);
    }
  }
}
```

시간 복잡도 분석

- 다른 예제
  - N이 주어질 때, $2^k \leq N$인 모든 양수 k에 대해, $\Sigma(2^k)$를 계산하는 프로그램
  - 코드에서 i는 $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \cdots \rightarrow 2^k \leq N$ 순으로 2의 거듭제곱대로 커짐
  - 따라서 최대 $\log_2 N$번만큼 반복되고, 시간 복잡도는 $O(\log N)$

```
int powSum(int N) {
  int result = 0;
  for (int i = 1; i <= N; i *= 2) {
    result += i;
  }
  return result;
}
```

시간 복잡도 분석

- 시간 복잡도는 문제를 풀 때 마다 계산하자.
  - 비효율적인 풀이의 경우 시간 초과가 날 가능성이 있음.
  - 느낌으로만 할 경우 이러한 “비효율적인 경우”를 인지하지 못함.
  - 보다 효율적인 문제해결을 위해 시간 복잡도를 계산하는 습관을 들이자.

Brute Force

2026-1학기 부경대학교 알고리즘 스터디

Brute Force

- 뜻: 난폭한 힘, 폭력
  - 무식하게 힘으로 해결한다는 뜻
- 모든 경우의 수를 확인하는 방법(알고리즘)을 뜻함.
  - ex) 4자리 숫자 자물쇠를 푸는 방법? -> 0000 ~ 9999 숫자를 전부 돌려본다.
- 이런 방법이 가능한 이유: 컴퓨터는 인간보다 연산이 빠르기 때문
  - 컴퓨터는 1초에 약 1억 회 연산이 가능함.
  - 물론 대놓고 무작정 이런 방법을 사용하면 안된다. (알고리즘이라는 것의 존재 이유를 생각해보자.)

블랙잭 BOJ 2798

- 문제
  - N장의 카드와 숫자 M이 주어졌을 때, 3장의 카드를 뽑아 M에 가장 가깝게 만드는 문제
  - $N \leq 100, M \leq 300{,}000$
  - 모든 경우의 수를 확인하는 방법을 생각해보자.

블랙잭 BOJ 2798

- 문제
  - N장의 카드와 숫자 M이 주어졌을 때, 3장의 카드를 뽑아 M을 넘지 않으면서 M에 가장 가깝게 만드는 문제
  - $N \leq 100, M \leq 300{,}000$
  - 모든 경우의 수를 확인하는 방법을 생각해보자.
- 풀이
  - 정답이 될 수 있는 경우의 수는 카드 세 장을 뽑는 경우의 수와 같음
  - 3중 for문을 통해 카드를 세 장 뽑아보자.
  - 시간 복잡도는 $O(n^3)$이므로, 1초 안에 충분히 돌아간다.

블랙잭 BOJ 2798

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
  ios_base::sync_with_stdio(false);
  cin.tie(nullptr), cout.tie(nullptr);

  int N, M, arr[100];
  cin >> N >> M;
  for (int i = 0; i < N; i++) cin >> arr[i];

  int maxVal = 0;
  for (int i = 0; i < N; i++) {
    for (int j = i + 1; j < N; j++) {
      for (int k = j + 1; k < N; k++) {
        if (arr[i] + arr[j] + arr[k] <= M) {
          maxVal = max(maxVal, arr[i] + arr[j] + arr[k]);
        }
      }     // min() : <algorithm> 라이브러리에 있는 최소값을 반환하는 함수
    }       // max() : <algorithm> 라이브러리에 있는 최대값을 반환하는 함수
  }

  cout << maxVal;
  return 0;
}
```

일곱 난쟁이 BOJ 2309

- 문제
  - 아홉 난쟁이의 키 $a_1 \sim a_9$가 주어질 때, 7명의 난쟁이를 뽑아 합이 100이 되도록 만들고 오름차순으로 출력하는 문제
  - $1 \leq i \leq 9, 1 \leq a_i < 100$
  - 블랙잭 문제처럼 7중 for문으로 풀이하면 될까? / 정렬은 어떻게 하지?

일곱 난쟁이 BOJ 2309

- 문제
  - 아홉 난쟁이의 키 $a_1 \sim a_9$가 주어질 때, 7명의 난쟁이를 뽑아 합이 100이 되도록 만들고 오름차순으로 출력하는 문제
  - $1 \le i \le 9, 1 \le a_i < 100$
  - 블랙잭 문제처럼 7중 for문으로 풀이하면 될까? / 정렬은 어떻게 하지?
- 풀이
  - 7명을 뽑는다 = 2명을 뽑지 않는다
  - 따라서 9명의 키의 합인 s를 정의하여 2중 for문으로 뽑지 않을 난쟁이($s - a_i - a_j = 100$인 i와 j)만 정해주면 됨.
  - 정렬은 <algorithm>에 정의된 sort()를 사용하여도 되지만, $a_i < 100$인 것을 이용하여 count 배열을 만들고 이를 토대로 크기가 9인 새 배열을 만들어 풀이 가능
  - n을 난쟁이의 수, m을 a_i의 최대크기라고 하면 시간 복잡도는 $O(n^2 + m)$

일곱 난쟁이 BOJ 2309

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
  ios_base::sync_with_stdio(false);
  cin.tie(nullptr), cout.tie(nullptr);

  // input
  int sum = 0, count[100]{}, arr[9];
  for (int i = 0; i < 9; i++) {
    int val;
    cin >> val;
    count[val]++, sum += val;
  }

  // sort
  int index = 0;
  for (int i = 0; i < 100; i++) {
    while (count[i] > 0) {
      arr[index] = i;
      count[i]--, index++;
    }
  }

  // get answer
  int ans1, ans2;
  for (int i = 0; i < 9; i++) {
    for (int j = i + 1; j < 9; j++) {
      if (sum - arr[i] - arr[j] == 100) ans1 = i, ans2 = j;
    }
  }

  // output
  for (int i = 0; i < 9; i++) {
    if (ans1 != i && ans2 != i) cout << arr[i] << '\n';
  }
  return 0;
}
```

분해합 BOJ 2231

- 문제
  - N의 최소 생성자를 구하는 문제 (없으면 0을 출력)
  - $M + (M$의 각 자릿수의 합$) = N$ 이면 "M은 N의 생성자"
  - $1 \leq N \leq 1{,}000{,}000$

분해합 BOJ 2231

- 문제
  - N의 최소 생성자를 구하는 문제 (없으면 0을 출력)
  - $M + (M$의 각 자릿수의 합$) = N$ 이면 "M은 N의 생성자"
  - $1 \leq N \leq 1{,}000{,}000$
- 풀이
  - N의 생성자 M은 항상 $M < N$을 만족함.
  - 따라서 $M + (M$의 각 자릿수의 합$) = N$ 을 만족하는 $M < N$을 구하면 됨.
  - 시간 복잡도는 $O(N)$

분해합 BOJ 2231

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
  ios_base::sync_with_stdio(false);
  cin.tie(nullptr), cout.tie(nullptr);

  int n, ans = 0;
  cin >> n;

  for (int i = n; i > 0; i--) {
    int m = i, k = i;
    while (k > 0) {
      m += k % 10;
      k /= 10;
    }
    if (m == n) ans = i;
  }

  cout << ans;
  return 0;
}
```

분해합2 BOJ 12348

- 문제
  - 이전 문제(분해합)에서 제한만 10^{18}으로 바뀐 문제
  - $1 \leq N \leq 1e18$
  - 1부터 하면 시간초과.. 범위를 줄일 수 있는 방법이 있을까?

분해합2 BOJ 12348

- 문제
  - 이전 문제(분해합)에서 제한만 10^{18}으로 바뀐 문제
  - $1 \le N \le 1e18$
  - 1부터 하면 시간초과.. 범위를 줄일 수 있는 방법이 있을까?
- 풀이
  - 생성자 M은 $N - 18 \cdot 9 \le M < N$의 값을 가진다.
  - 자릿수는 최대 18개이고, 각각 최대 9의 값을 가지기 때문
  - 따라서 N보다 작은 수들 중 가장 큰 $18 \cdot 9$ 개의 값만 확인하면 된다.
  - 시간 복잡도는 $O(18 \cdot 9) = O(1)$

분해합2 BOJ 12348

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
  ios_base::sync_with_stdio(false);
  cin.tie(nullptr), cout.tie(nullptr);

  long long n, ans = 0;  // int의 범위는 약 20억이므로 long long을 써야함
  cin >> n;

  for (long long i = n; i >= n - 18 * 9; i--) {
    long long m = i, k = i;
    while (k > 0) {
      m += k % 10;
      k /= 10;
    }
    if (m == n) ans = i;
  }

  cout << ans;
  return 0;
}
```

그 외 연습문제

- 1436 영화감독 숌
- 7568 덩치
- 19532 수학은 비대면강의입니다
- 4673 셀프 넘버
- 1057 토너먼트
- 1018 체스판 다시 칠하기